

Módulo V “Desarrolla aplicaciones para dispositivos móviles”

Submódulo I “Desarrolla aplicaciones móviles para Android”.

--- Parte 3 ---

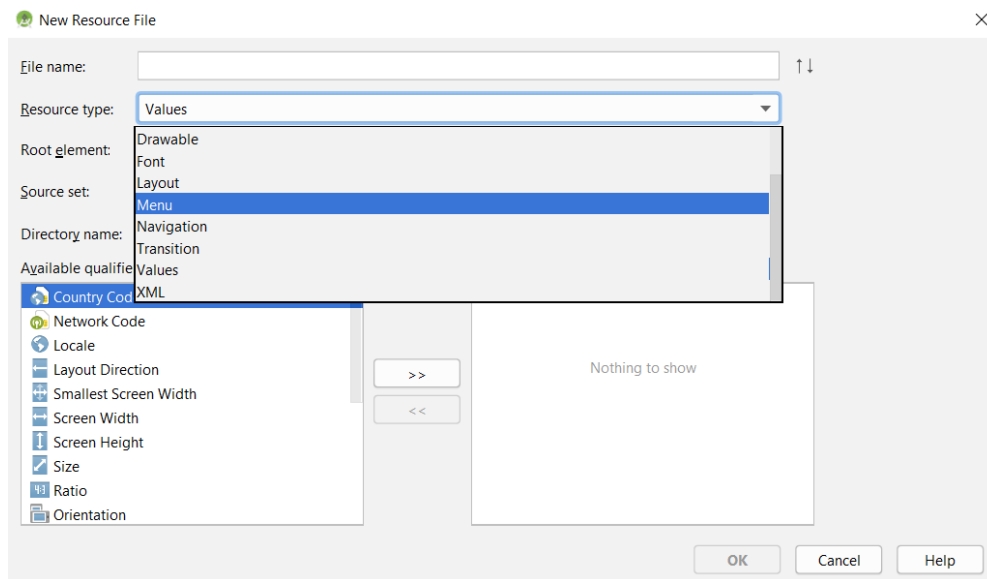
INTENT IMPLÍCITO MENÚ

Para complementar la app, le agregaremos la función de acceder a los Contactos registrados en nuestro dispositivo mediante un **menú desplegable** para que el usuario acceda a la opción y dirigirlo a dicha función.

Existen varias formas de hacer estos menús, una de ellas será la siguiente forma en la que trabajaremos.

Buscamos la carpeta res y hacemos clic derecho New->Android resource file, abriéndonos la siguiente ventana:

En Resource type buscamos menu, y en File name tecleamos el nombre del menú.



Esto ocasiona que nos cree una carpeta llamada menu y un archivo dentro de ella, llamado menu.xml, el cual abrimos y tecleamos lo siguiente:

```
<item
    android:id="@+id/menuContactos"
    android:title="Contactos"
    app:showAsAction="never"
/>
```

Donde **item** nos permite agregar los elementos cuando estamos creando un menú, el **id** nos sirve para trabajar desde código con el menú, el **title** es para que cuando el usuario despliegue los tres puntos, pueda ver la opción, finalizamos con **showAsAction**, que permite darle una acción al menú para que el dispositivo sepa cómo va a manejar la vista del menú, mostrando diferentes opciones, las cuales son: **never**, **always**, **collapseActionView**, **ifRoom** y **withText**.

La implementación del menú se hará en el ThirdActivity.kt, pero ¿cómo se hace? Lo primero será sobrescribir 2 métodos, **onCreateOptionsMenu** y **onOptionsItemSelected**. Al primer método le estará llegando un menú de tipo menú, además le tenemos que pasar la vista, es decir, en dónde se encuentra este menú, siendo la instrucción **menuInflater.inflate(R.menu.menu, menu)** (ubicación, parámetro). Terminamos con un **return true**.

La sobreescritura del siguiente método es **onOptionsItemSelected**, al cual le indicamos que cuando el usuario presione la opción, sepa qué comportamiento realizará. Usamos **when(item.itemId)**, donde item llega vía parámetro. Entonces cuando el usuario toque la opción del menú, **R.id.menuContactos** mandará el intent que estamos queriendo enlazar. Donde intentContactos será igual a la clase Intent, le pasamos el **ACTION_VIEW** y su respectivo **uri**, en el cual **content://contacts/people** redirigirá a la aplicación donde almacenamos los contactos. El código completo queda de la siguiente manera:

```
override fun onCreateOptionsMenu(menu : Menu):Boolean{
    menuInflater.inflate(R.menu.menu,menu)
    return true
}

override fun onOptionsItemSelected(item: MenuItem?): Boolean {
    when(item.itemId){
        R.id.menuContactos -> {
            val intentContactos = Intent(Intent.ACTION_VIEW, Uri.parse("content://contacts/people"))
            startActivity(intentContactos)
        }
    }
    return super.onOptionsItemSelected(item)
}
```

Se pueden agregar íconos dentro del menú. Para esto, un ítem del menú será el principal, mientras que los otros 3 elementos del menú se agruparán en uno solo menú. Además, debemos agregar algunos **Image Asset** para generar los íconos que estarán acompañando el texto dentro del menú, buscándolos como contacts, sms y video. Los íconos se agregan mediante la propiedad **android:icon**. El código dentro del XML del menú, queda de la siguiente manera:

```
<item
    android:id="@+id/menuPrincipal"
    android:title="Opciones"
    app:showAsAction="always">

    <menu>
        <item
            android:id="@+id/menuContactos"
            android:icon="@mipmap/ic_contacts"
            android:title="Contactos"
            app:showAsAction="ifRoom" />
        <item
            android:id="@+id/menuMensaje"
            android:icon="@mipmap/ic_sms"
            android:title="Enviar SMS"
            app:showAsAction="ifRoom" />
        <item
            android:id="@+id/menuVideo"
            android:icon="@mipmap/ic_video"
            android:title="Video"
            app:showAsAction="ifRoom" />
    </menu>
</item>
```

En la parte lógica del menú implementada en el ThirdActivity, accederemos mediante los id que se les asignaron. El primero a trabajar es menuMensaje:

```
R.id.menuMensaje -> {  
    val intentSMS = Intent()  
    intentSMS.action = Intent.ACTION_SENDTO  
    intentSMS.data = Uri.parse("smsto:")  
    //intentSMS.putExtra("address", "8341381216")  
    intentSMS.putExtra("sms_body", "Cuerpo del SMS desde un menú")  
    startActivity(intentSMS)  
}
```

Para iniciar un mensaje de texto SMS, se usa una de las acciones como **ACTION_SENDTO** y especificamos los detalles del mensaje, como el número de teléfono y el cuerpo del mensaje.

Enseguida, podemos trabajar también con menuVideo:

```
R.id.menuVideo -> {  
    val intentVideo = Intent("android.media.action.VIDEO_CAPTURE")  
    startActivity(intentVideo)  
}
```

Se trabaja de manera muy similar al **ImageButtonCamera**.

ACTIONBAR CON FLECHA ATRÁS

Para colocar en la barra de estado de la parte superior de la pantalla la opción de regresar al activity anterior, se deberán agregar las siguientes líneas:

```
supportActionBar!!.title = "Atrás"  
supportActionBar!!.setDisplayHomeAsUpEnabled(true)
```



Es importante mencionar que esas líneas únicamente permiten visualizar la flecha y la palabra de atrás, pero hay un paso adicional para que funcione dicha flecha de atrás. Para ello nos iremos al **AndroidManifest.xml** y buscaremos el activity al cual queremos hacerle esa adecuación, en nuestro caso, `.SecondActivity` o `ThirdActivity`.

```
<activity android:name=".ThirdActivity" android:parentActivityName=".MainActivity">  
<meta-data android:name="android.support.PARENT_ACTIVITY" android:value=".MainActivity"/>  
</activity>
```

La propiedad **parentActivityName** enlaza del activity en donde estamos al main activity (o al que se quiera mandar).